

LECTURE 8

FUNCTIONS WITH ARRAY INPUT/OUTPUT, PASS BY VALUE VS PASS BY REFERENCE,
GLOBAL & STATIC VARIABLES,
RECURSION

RECAP

- LIKE WE DECLARE TYPE OF MATH FUNCTION: E.G., $f: \mathbb{R}^n \mapsto \mathbb{R}$
 - WE NEED TO DECLARE A C FUNCTION, BEFORE IT'S FIRST USE
 - OUTSIDE MAIN (OUTSIDE ANY FUNCTION)
 - *output_datatype* function_name(*input1_datatype*, *input2_datatype*);
- STANDARD LIBRARY FUNCTIONS STORE THIS DECLARATION IN HEADER FILE

RECAP

- CODE OF FUNCTION IS WRITTEN AS SEPARATE BLOCK CALLED DEFINITION
- *out_type* function_name(*in1_type* var1, *in2_type* var2){
 block-of-code
}
- OUTSIDE ANY FUNCTION (INCLUDING MAIN)
- ANY ORDER IS FINE, SEPARATE FILE ALSO FINE
 - PROVIDE ALL FILES TOGETHER TO *gcc*

RECAP

- *return* IS USED IN TWO WAYS
 - EXPRESSION AFTER *return* IS EVALUATED AND COPIED AS VALUE OF FUNCTION IN CALLER CODE
 - RETURNING CONTROL TO CALLER CODE E.G., MULTIPLE *return* IN IF-ELSE BLOCKS

RECAP

- FUNCTIONS HELP IN
 - SOLVING COMPLEX PROBLEMS BY DIVIDING CODE INTO SMALLER COMPONENTS
 - MODULARIZING CODE FOR EASE IN READABILITY AND DEBUGGING
 - CODE REUSE/DISTRIBUTION
 - REPRESENT MATHEMATICAL/LOGICAL FUNCTIONS IN C
 - EASILY IMPLEMENTING RECURSIVE FORMULAE (WILL BE TAUGHT TODAY)

RECAP

- FUNCTIONS HELP IN
 - SOLVING COMPLEX PROBLEMS BY DIVIDING CODE INTO SMALLER COMPONENTS
 - MODULARIZING CODE FOR EASE IN READABILITY AND DEBUGGING
 - CODE REUSE/DISTRIBUTION
 - REPRESENT MATHEMATICAL/LOGICAL FUNCTIONS IN C
 - EASILY IMPLEMENTING RECURSIVE FORMULAE (WILL BE TAUGHT TODAY)

RECAP

- COMPILER FACILITATES “LINKING” VARIOUS FUNCTIONS
- CPU FACILITATES NESTED FUNCTION CALLING USING STACK DATA STRUCTURE

STACK FRAMES

- EACH FUNCTION CALL CREATES A **STACK FRAME** THAT HOLDS:
 - PARAMETERS
 - LOCAL VARIABLES
 - RETURN ADDRESS (WHERE TO RETURN AFTER THE FUNCTION IS DONE)

STACK FRAMES

```
def add(a, b):  
    result = a + b  
    return result
```

```
def main():  
    x = 5  
    y = 10  
    sum = add(x, y)  
    print(sum)
```

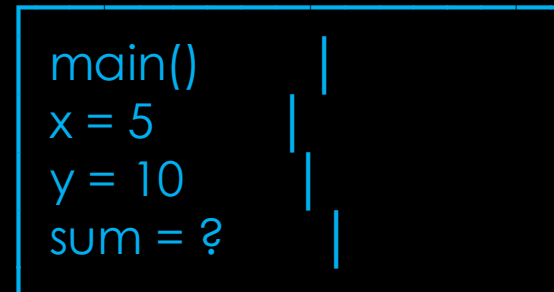
```
main()
```

STACK FRAMES

```
def add(a, b):  
    result = a + b  
    return result
```

```
def main():  
    x = 5  
    y = 10  
    sum = add(x, y)  
    print(sum)
```

main()



STACK FRAMES

```
def add(a, b):  
    result = a + b  
    return result
```

```
def main():  
    x = 5  
    y = 10  
    sum = add(x, y)  
    print(sum)
```

main()

```
add(a=5, b=10) |  
result = ?    |
```

```
main() |  
x = 5  |  
y = 10 |  
sum = ? |
```

STACK FRAMES

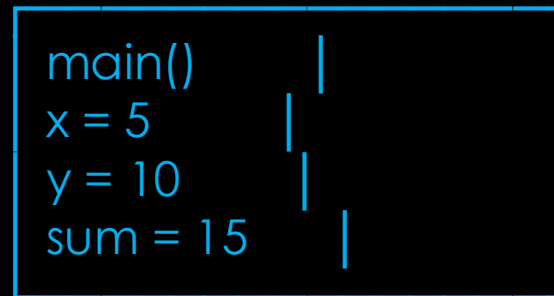
```
def add(a, b):  
    result = a + b  
    return result
```

```
def main():  
    x = 5  
    y = 10  
    sum = add(x, y)  
    print(sum)
```

main()

[return 15 to main()]

(add frame is removed)



PASSING ARRAYS AS INPUT

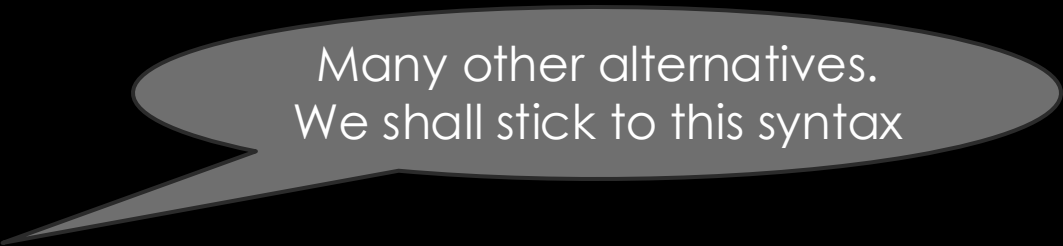
TO FUNCTIONS

SYNTAX

```
void do_something(int size, double vector[size]){    //Definition  
    block_of_code  
}
```

```
// Declaration without block ended with ; is a valid declaration
```

SYNTAX



Many other alternatives.
We shall stick to this syntax

```
void do_something(int size, double vector[size]){    //Definition  
    block_of_code  
}
```

```
// Declaration without block ended with ; is a valid declaration
```

EXAMPLE: DOT-PRODUCT OF VECTORS

$$a^{\top} b \equiv \sum_{i=1}^n a_i b_i$$

DOT-PRODUCT FUNCTION - DEFINITION

```
double dotproduct(int dimension,double vector1[dimension],double vector2[dimension]){  
    double sum=0;  
  
    for(int i=0;i<dimension;i++)  
        sum+=vector1[i]*vector2[i];  
  
    return sum;  
}
```

DOT-PRODUCT FUNCTION - DECLARATION

```
double dotproduct(int dimension, double vector1[dimension], double vector2[dimension]);
```

FROBENIUS INNER-PRODUCT

$$\langle A, B \rangle_F \equiv \text{trace}(A^\top B) = \sum_{i=1}^m \sum_{j=1}^n A_{ij} B_{ij}$$

FROBENIUS INNER-PRODUCT FUNCTION

```
double innerproduct(int m, int n, double matrix1[m][n],double matrix2[m][n]);
```



FROBENIUS INNER-PRODUCT FUNCTION

```
double innerproduct(int m, int n, double matrix1[m][n],double matrix2[m][n]){
```

```
}
```

FROBENIUS INNER-PRODUCT FUNCTION

```
double innerproduct(int m, int n, double matrix1[m][n],double matrix2[m][n]){  
    double sum=0;  
  
    for(int i=0;i<m;i++)  
        for(int j=0;j<n;j++)  
            sum+=matrix1[i][j]*matrix2[i][j];  
  
    return sum;  
}
```

WHAT HAPPENS?

```
double innerproduct(double matrix1[m][n], double matrix2[m][n], int m, int n);
```

WHAT HAPPENS?

```
double innerproduct(double matrix1[m][n], double matrix2[m][n], int m, int n);
```

Identifier
undefined

Identifier
undefined

HOW TO CALL ?

```
#include <stdio.h>

double innerproduct(int m, int n, double matrix1[m][n],double matrix2[m][n]);

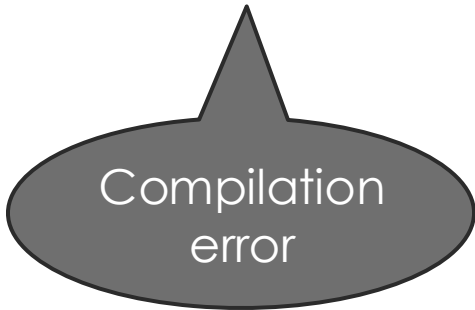
int main(void){
    double my_matrix1[5][20]={1},my_matrix2[5][20]={2},ip;
    printf("%lf",innerproduct(5,20,my_matrix1[5][20],my_matrix2[5][20]));
    return 0;
}
```

HOW TO CALL ?

```
#include <stdio.h>

double innerproduct(int m, int n, double matrix1[m][n], double matrix2[m][n]);

int main(void){
    double my_matrix1[5][20]={1}, my_matrix2[5][20]={2}, ip;
    printf("%lf", innerproduct(5, 20, my_matrix1[5][20], my_matrix2[5][20]));
    return 0;
}
```



Compilation
error

CALLER MAIN.C

```
#include <stdio.h>

double innerproduct(int m, int n, double matrix1[m][n],double matrix2[m][n]);

int main(void){
    double my_matrix1[5][20]={1},my_matrix2[5][20]={2};
    printf("%lf",innerproduct(5,20,my_matrix1,my_matrix2));
    return 0;
}
```

WHAT IS ACTUALLY PASSED ?

```
#include <stdio.h>

double innerproduct(int m, int n, double matrix1[m][n], double matrix2[m][n]);

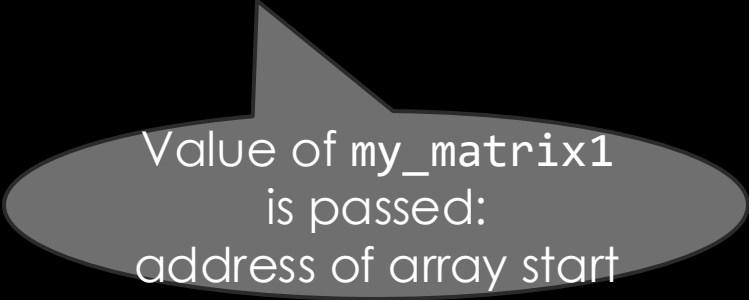
int main(void){
    double my_matrix1[5][20]={1}, my_matrix2[5][20]={2};
    printf("%lf", innerproduct(5, 20, my_matrix1, my_matrix2));
    return 0;
}
```

WHAT IS ACTUALLY PASSED ?

```
#include <stdio.h>

double innerproduct(int m, int n, double matrix1[m][n], double matrix2[m][n]);

int main(void){
    double my_matrix1[5][20]={1}, my_matrix2[5][20]={2};
    printf("%lf", innerproduct(5, 20, my_matrix1, my_matrix2));
    return 0;
}
```



Value of my_matrix1
is passed:
address of array start

PASSING ARRAYS: BEHIND THE SCENES

- VALUE OF `my_matrix1` IS PASSED TO FUNCTION
- THIS VALUE IS AN ADDRESS IN MAIN'S STACK FRAME
- VALUE/ADDRESS IS COPIED INTO LOCAL
- INDEXING INSIDE FUNCTION IS SAME AS INDEXING INTO IN MAIN'S STACK FRAME
 - SO SAME ARRAY VALUES ARE AVAILABLE WITHOUT COPYING!
 - THE IDENTIFIER IN MAIN IS NOT AVAILABLE TO FUNCTION (DOESN'T MATTER)
- ARRAY DECLARATION INSIDE FUNCTION PARAMETERS—NO SPACE ALLOCATION FOR ELEMENTS

PASSING ARRAYS: BEHIND THE SCENES

- VALUE OF `my_matrix1` IS PASSED TO FUNCTION
- THIS VALUE IS AN ADDRESS IN MAIN'S STACK FRAME
- VALUE/ADDRESS IS COPIED INTO LOCAL
- INDEXING INSIDE FUNCTION IS SAME AS INDEXING INTO IN MAIN'S STACK FRAME
 - SO SAME ARRAY VALUES ARE AVAILABLE WITHOUT COPYING!
 - THE IDENTIFIER IN MAIN IS NOT AVAILABLE TO FUNCTION (DOESN'T MATTER)
- ARRAY DECLARATION INSIDE FUNCTION PARAMETERS—NO SPACE ALLOCATION FOR ELEMENTS

PASSING ARRAYS: BEHIND THE SCENES

- VALUE OF `my_matrix1` IS PASSED TO FUNCTION
- THIS VALUE IS AN ADDRESS IN MAIN'S STACK FRAME
- VALUE/ADDRESS `my_matrix1` IS COPIED INTO LOCAL `matrix1`
- INDEXING INSIDE FUNCTION IS SAME AS INDEXING INTO IN MAIN'S STACK FRAME
 - SO SAME ARRAY VALUES ARE AVAILABLE WITHOUT COPYING!
 - THE IDENTIFIER IN MAIN IS NOT AVAILABLE TO FUNCTION (DOESN'T MATTER)
- ARRAY DECLARATION INSIDE FUNCTION PARAMETERS—NO SPACE ALLOCATION FOR ELEMENTS

PASSING ARRAYS: BEHIND THE SCENES

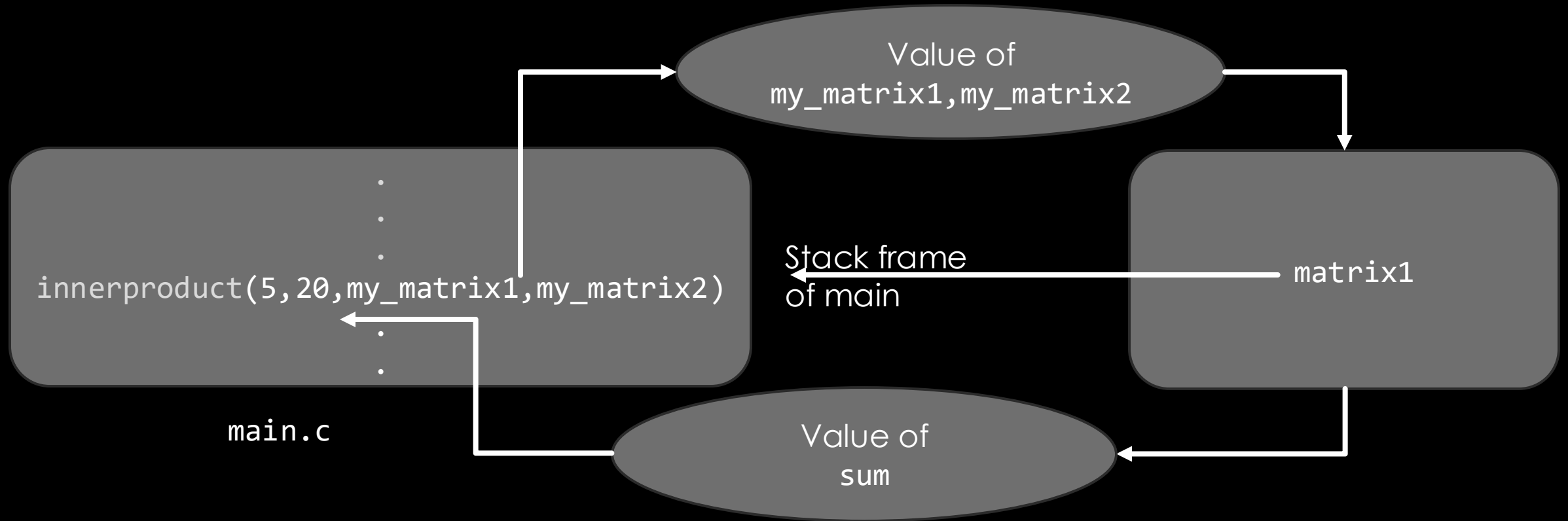
- VALUE OF `my_matrix1` IS PASSED TO FUNCTION
- THIS VALUE IS AN ADDRESS IN MAIN'S STACK FRAME
- VALUE/ADDRESS `my_matrix1` IS COPIED INTO LOCAL `matrix1`
- INDEXING `matrix1` INSIDE FUNCTION IS SAME AS INDEXING INTO `my_matrix1` IN MAIN'S STACK FRAME
 - SO SAME ARRAY VALUES ARE AVAILABLE WITHOUT COPYING!
 - THE `my_matrix1` IDENTIFIER IN MAIN IS NOT AVAILABLE TO FUNCTION (DOESN'T MATTER)
- ARRAY DECLARATION INSIDE FUNCTION PARAMETERS—NO SPACE ALLOCATION FOR ELEMENTS

PASSING ARRAYS: BEHIND THE SCENES

- VALUE OF `my_matrix1` IS PASSED TO FUNCTION
- THIS VALUE IS AN ADDRESS IN MAIN'S STACK FRAME
- VALUE/ADDRESS `my_matrix1` IS COPIED INTO LOCAL `matrix1`
- INDEXING `matrix1` INSIDE FUNCTION IS SAME AS INDEXING INTO `my_matrix1` IN MAIN'S STACK FRAME
 - SO SAME ARRAY VALUES ARE AVAILABLE WITHOUT COPYING!
 - THE `my_matrix1` IDENTIFIER IN MAIN IS NOT AVAILABLE TO FUNCTION (DOESN'T MATTER)
- ARRAY DECLARATION INSIDE FUNCTION PARAMETERS—NO SPACE ALLOCATION FOR ELEMENTS



VISUALIZING a.out



WHAT HAPPENS?

```
#include <stdio.h>

void something(int m, double vector[m]){
    vector[0]=1;
}

int main(void){
    double my_vector[5] ={0};
    something(5,my_vector);
    printf("%.01f",my_vector[0]);
}
```

WHAT HAPPENS?

```
#include <stdio.h>

void something(int m, double vector[m]){
    vector[0]=1;
}

int main(void){
    double my_vector[5] ={0};
    something(5,my_vector);
    printf("%.01f",my_vector[0]);    // 1
}
```

WHAT HAPPENS?

```
#include <stdio.h>

void something(int m){
    m=1;
}

int main(void){
    int m=0;
    something(m);
    printf("%d",m);
}
```

WHAT HAPPENS?

```
#include <stdio.h>

void something(int m){
    m=1;
}

int main(void){
    int m=0;
    something(m);
    printf("%d",m);    // 0
}
```

const QUALIFIER

- PROTECTS ARRAYS IN CALLER BEING MODIFIED BY CALLED FUNCTIONS ACCIDENTALLY
- `void do_something(int m, const double vector[m])`
- COMPILER GIVES ERROR IF CALLED FUNCTION MODIFIES ELEMENTS OF `vector`

PASSING OBJECTS/VARIABLES BY PASSING REFERENCE

C ALWAYS PASSES BY VALUE!

PASSING REFERENCE USING ARRAYS

```
#include <stdio.h>

void something(int m, double number[m]){ number[0]++; }

int main(void){
    double my_number[1]={0};
    something(1,my_number);
    printf("%lf",my_number[0]);    // 1 is printed
    return 0;
}
```

PASSING REFERENCE USING ARRAYS

```
#include <stdio.h>

void something(double number[]){ number[0]++; }

//Alternative syntax 1-d arrays

int main(void){
    double my_number[1]={0};
    something(my_number);
    printf("%lf",my_number[0]);
    return 0;
}
```

SWAPPING

```
#include <stdio.h>

void swap(double number1[], double number2[]){
    double temp=number1[0];number1[0]=number2[0];number2[0]=temp;}

int main(void){
    double my_number1[1]={1}, my_number2[1]={2};
    swap(my_number1,my_number2);
    printf("%lf,%lf",my_number1[0],my_number2[0]);    // 2,1 is printed
    return 0;
}
```

SWAPPING – WRONG CODE

```
#include <stdio.h>

void swap(double number1, double number2){
double temp=number1;number1=number2;number2=temp;}

int main(void){
    double my_number1=1, my_number2=2;
    swap(my_number1,my_number2);
    printf("%lf,%lf",my_number1,my_number2);    // 1,2 is printed
    return 0;
}
```

PASSING VALUE VS PASSING REFERENCE

- C FUNCTIONS ALWAYS PASS BY VALUE
- BUT WHEN VALUE IS AN ADDRESS – PASSING OBJECTS IS SIMULATED
- PASSING BY VALUE CANNOT NOT CHANGE VALUES OF CALLER VARIABLES
- PASSING BY REFERENCE CAN CHANGE VALUES OF CALLER VARIABLES

PASSING REFERENCE – WITHOUT ARRAYS

- PASSING ADDRESSES IS KNOWN TO US: E.G., `swap(&my_number1, &my_number2);`
`// double my_number1, my_number2;`
- BUT HOW TO DECLARE/DEFINE SUCH FUNCTIONS WITH INPUTS AS ADDRESSES?
- NEXT LECTURE SYNTAX FOR IDENTIFIERS HOLDING ADDRESSES WILL BE TAUGHT – POINTERS
 - DEFINE FUNCTIONS WITH INPUT AS POINTERS
 - PASSING POINTERS IS CONVENIENT FOR PASSING BY REFERENCE THAN PASSING ARRAYS

PASSING REFERENCE – WITHOUT ARRAYS

- PASSING ADDRESSES IS KNOWN TO US: E.G., `swap(&my_number1, &my_number2);`
`// double my_number1, my_number2;`
- BUT HOW TO DECLARE/DEFINE SUCH FUNCTIONS WITH INPUTS AS ADDRESSES?
- NEXT LECTURE SYNTAX FOR IDENTIFIERS HOLDING ADDRESSES WILL BE TAUGHT – POINTERS
 - DEFINE FUNCTIONS WITH INPUT AS POINTERS
 - PASSING POINTERS IS CONVENIENT FOR PASSING BY REFERENCE THAN PASSING ARRAYS

PASSING ARRAYS AS OUTPUT

FROM FUNCTIONS

ARRAYS AS OUTPUT

- EQUIVALENT TO RETURNING ADDRESS OF AN ARRAY PERHAPS STORED INSIDE FUNCTION STACK FRAME
 - MEMORY FREED ONCE STACK POPPED DUE TO FUNCTION RETURN
 - RESULTS IN INVALID ADDRESS
- MAKES SENSE IF ARRAY IS LOCATED IN CALLER'S STACK FRAME (OR EVEN BELOW; MAIN'S FRAME)
 - SEND ARRAY AS INPUT RATHER THAN OUTPUT
- RETURNING POINTERS (TAUGHT IN NEXT LECTURE)

ARRAYS AS OUTPUT

- EQUIVALENT TO RETURNING ADDRESS OF AN ARRAY PERHAPS STORED INSIDE FUNCTION STACK FRAME
 - MEMORY FREED ONCE STACK POPPED DUE TO FUNCTION RETURN
 - RESULTS IN INVALID ADDRESS
- MAKES SENSE IF ARRAY IS LOCATED IN CALLER'S STACK FRAME (OR EVEN BELOW; MAIN'S FRAME)
 - SEND ARRAY AS INPUT RATHER THAN OUTPUT
- RETURNING POINTERS (TAUGHT IN NEXT LECTURE)

ARRAYS AS OUTPUT

- EQUIVALENT TO RETURNING ADDRESS OF AN ARRAY PERHAPS STORED INSIDE FUNCTION STACK FRAME
 - MEMORY FREED ONCE STACK POPPED DUE TO FUNCTION RETURN
 - RESULTS IN INVALID ADDRESS
- MAKES SENSE IF ARRAY IS LOCATED IN CALLER'S STACK FRAME (OR EVEN BELOW; MAIN'S FRAME)
 - SEND ARRAY AS INPUT RATHER THAN OUTPUT
- RETURNING POINTERS (TAUGHT IN NEXT LECTURE)

EXAMPLE: OUTER-PRODUCT OF VECTORS

$$a \otimes b \equiv \begin{bmatrix} a_1 b_1 & \cdots & a_1 b_m \\ \vdots & \ddots & \vdots \\ a_n b_1 & \cdots & a_n b_m \end{bmatrix}$$

OUTER-PRODUCT FUNCTION – OUTPUT AS INPUT

```
void outerproduct(int m,double vector1[m],int n,double vector2[n],double matrix[m][n])
{
    for(int i=0;i<m;i++)
        for(int j=0;j<n;j++)
            matrix[i][j]=vector1[i]*vector2[j];

    return;
}
```

OUTER-PRODUCT FUNCTION - ALTERNATIVE

```
#include <stdio.h>

double matrix[50][100];          //Global variables

void outerproduct(int m,double vector1[m],int n,double vector2[n]){
    for(int i=0;i<m;i++)
        for(int j=0;j<n;j++)
            matrix[i][j]=vector1[i]*vector2[j];}

int main(void){
    double my_vector1[50]={1},my_vector2[100]={2};
    outerproduct(50,my_vector1,100,my_vector2);
}
```

STORAGE CLASSES

GLOBAL AND STATIC VARIABLES

GLOBAL VARIABLES

- DECLARED OUTSIDE FUNCTIONS
- SCOPE IS ANY FUNCTION/BLOCK BELOW IT
- ALLOCATED ONCE (STATICALLY) AT COMPILE TIME
 - PART OF OBJECT FILE (NOT STACK FRAME)
 - LIFETIME – BEGINNING OF PROGRAM TO END
- CANNOT DECLARE VARIABLE-SIZED ARRAYS

GLOBAL VARIABLES – USE CASES

- INSTEAD OF RETURNING ARRAYS FROM FUNCTIONS
- STORING DATA ONCE, PROCESSED BY MULTIPLE FUNCTIONS
 - SORTING, SEARCHING ROUTINES
- EFFICIENT CODES (BUT HARDER TO DEBUG/SECURE)

STATIC LOCAL VARIABLES

- DECLARED INSIDE FUNCTIONS
- `static int dimension=2; // SYNTAX, initialized only once!`
- DEFAULT INITIALIZATION TO ZERO
- SCOPE IS LOCAL - ANY FUNCTION/BLOCK NESTED IT
- ALLOCATED ONCE (STATICALLY) AT COMPILE TIME
 - PART OF OBJECT FILE (NOT STACK FRAME)
 - LIFETIME – BEGINNING OF PROGRAM TO END
- CANNOT DECLARE VARIABLE-SIZED ARRAYS

STATIC LOCAL VARIABLES – USE CASES

- RETURNING ARRAY ADDRESSES (POINTERS) FROM FUNCTIONS
 - RETURNED ADDRESS REMAINS VALID
- MAINTAINING COUNTERS ACROSS MULTIPLE FUNCTION CALLS
- AS SECURE VERSIONS OF GLOBAL VARIABLES

TYPES OF ALLOCATION

- STATIC ALLOCATION
 - AT COMPILE TIME, PART OF OBJECT FILE
 - FIXED-SIZED ARRAYS ETC.
- DYNAMIC ALLOCATION
 - AT RUN-TIME, ON STACK FRAME
 - VARIABLE-SIZE ARRAYS ETC.
- `malloc` (TAUGHT NEXT LECTURE)
 - AT RUN-TIME BUT NOT ON STACK FRAME (ON HEAP)
 - VARIABLE-SIZE ARRAYS ETC.
 - RETURN VALID ADDRESS (POINTERS)

RECURSION

PINGALA NUMBERS

$$P_n \equiv P_{n-1} + P_{n-2}, n \geq 3$$

$$P_1 \equiv 1, P_2 \equiv 2$$

PINGALA NUMBERS - RECURSION

```
unsigned int pingala(unsigned int n)
{
    if((n==1) || (n==2))           // Base case
        return (n==1)?1:2;
    else
        return pingala(n-1)+pingala(n-2); // Recursive definition
}
```

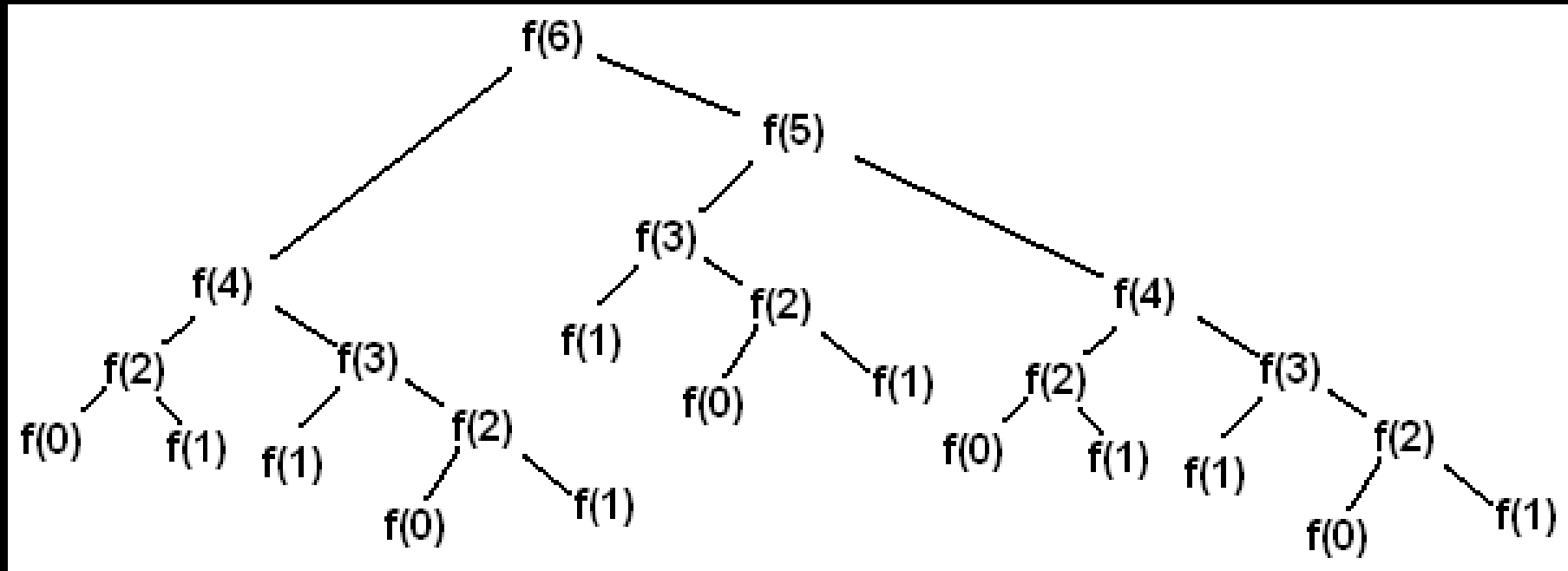

MAIN.C - RECURSION

```
#include <stdio.h>

unsigned int pingala(unsigned int n);

int main(void){
    unsigned int number; printf("Enter number of beats:");scanf("%d", &number);
    printf("Number of verse styles=%d", pingala(number));
}
```

RECURSIVE CALLS - VISUALIZATION



FACTORIAL – RECURSIVE FORMULA

$$n! = n \times (n - 1)!, n \geq 1$$

$$0! = 1$$

FACTORIAL - RECURSION

```
unsigned long int factorial(unsigned int n)
{
    if(n==0)                // Base case
        return 1;
    else
        return n*factorial(n-1); // Recursive definition
}
```

MAIN.C - RECURSION

```
#include <stdio.h>

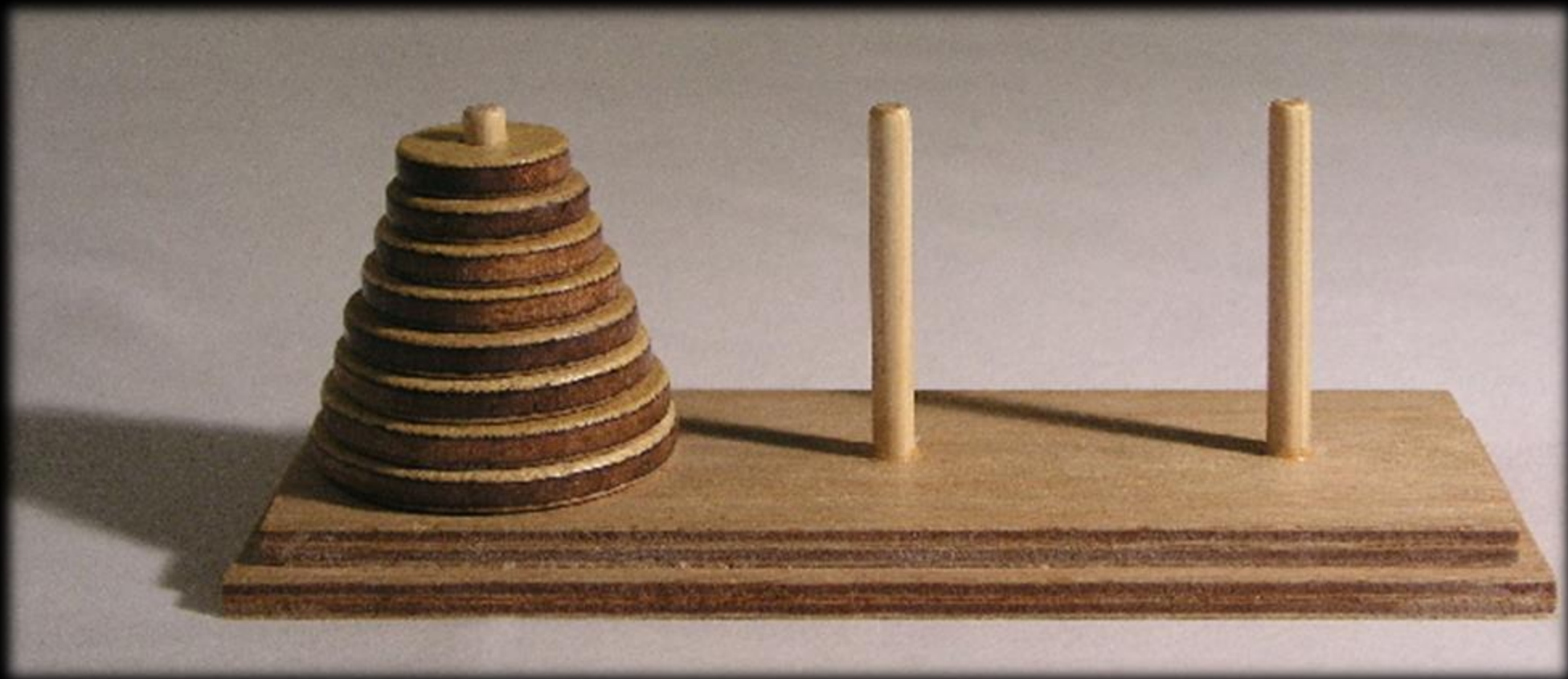
unsigned long int factorial(unsigned int n);

int main(void){
    unsigned int number; printf("Enter number:");scanf("%u", &number);
    printf("%u!=%lu", number,factorial(number));
}
```

TOUR SACRÉE DE BRAHMA

D'APRÈS UNE VIEILLE LÉGENDE INDIENNE, LES BRAHMES SE SUCCÈDENT DEPUIS BIEN LONGTEMPS, SUR LES MARCHES DE L'AUTEL, DANS LE **TEMPLE DE BÉNARÈS**, POUR EXÉCUTER LE DÉPLACEMENT DE LA **TOUR SACRÉE DE BRAHMA**, AUX SOIXANTE-QUATRE ÉTAGES EN OR FIN, GARNIS DE DIAMENTS DE **GOLCONDE**. QUANT TOUT SERA FINA, LA TOUR ET LES BRAHMES TOMBERONT, ET CE SERA LA FIN DU MONDE !

TOWERS OF BRAHMA



TOWERS OF BRAHMA



TOWERS OF BRAHMA - RECURSION

- MOVE N DISKS FROM PEG1 TO PEG3:
 - MOVE N-1 DISKS FROM PEG1 TO PEG2, USING PEG3 AS TEMP
 - MOVE 1 DISK FROM PEG1 TO PEG3
 - MOVE N-1 DISKS FROM PEG2 TO PEG3, USING PEG1 AS TEMP

TOWERS OF BRAHMA

```
#include <stdio.h>

void move(unsigned int n, unsigned short int from, unsigned short int to, unsigned short int via ){
    if(n==1)
        printf("%hu -> %hu\n", from, to);
    else{
        move(n-1,from,via,to);
        move(1,from,to,via);
        move(n-1,via,to,from);}}

int main(){
    unsigned int no_disks;scanf("%u",&no_disks);
    move(no_disks,1,3,2);
}
```

ITERATION VS RECURSION

- TYPICALLY, RECURSION IS INEFFICIENT
 - OVERHEAD OF STACK, FRAME DATA
 - UN-NECESSARY REPEATED COMPUTATION
- THERE ARE PROBLEMS WHERE ITERATION ALGORITHM IS NOT OBVIOUS
 - ITERATION IS DIFFICULT TO CODE/DEBUG
- FUNCTIONAL PROGRAMMING — MORE EFFICIENT RECURSION